

Style Transfer



Leon Gatys, Alexander Ecker, Matthias Bethge; A Neural Algorithm of Artistic Style .*Journal of Vision* 2016;16(12):326

Style Transfer

- A computer vision technique that allows to recompose content of an image in the style of another
- Takes two images—a *content image* and a *style reference image*—and blends them together
- Resulting output image retains core elements of *content* image, but appears to be “painted” in the style of the *style* reference image
- Style transfer can be used in the following areas:
 - Photo and video editors
 - Commercial art
 - Gaming
 - Virtual Reality

(a)Content image

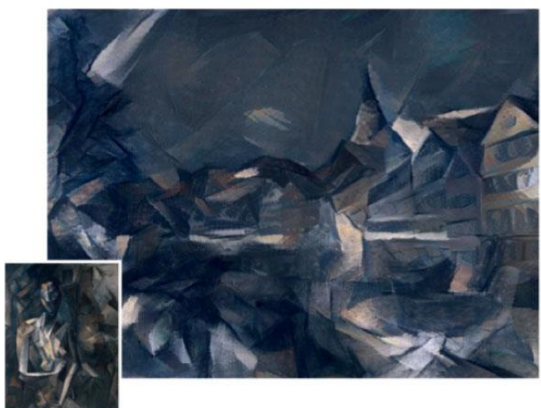


(b)Style image



(c)Generated image





Try – take your selfie as the content image and a Van Gogh painting as a style reference

Modes and types of style transfer

- Traditional supervised learning approach:
 - Learned style transfer requires a pair of input images - an original image and an artistic representation of that original image
 - A model learns the transformation and can apply it to new original images
 - This approach largely impractical, as these kinds of image pairs rarely exist
- In recent years, **neural style transfer (NST)**, has changed this

How does style transfer work?

- NST employs deep neural networks to power the transformations
- Neural networks used to extract statistical features of images related to content and style
 - Can quantify how well style transfer is working without explicit image pairs
 - Only a single style reference image is needed for neural network to apply it to original content images
- Training a style transfer model requires two networks:
 - A pre-trained feature extractor and a transfer network

How does style transfer work?

- Tendency for individual layers of deep CNNs to specialize in understanding specific features of an image
 - Some layers learn to extract content of an image (shape of a dog or position of a car)
 - While others learn to focus on texture (the small brush strokes of a painter or the fractal patterns of nature)

- Recall, what CNNs typically learn:

Content representation: feature maps often show spatial structure without texture/style

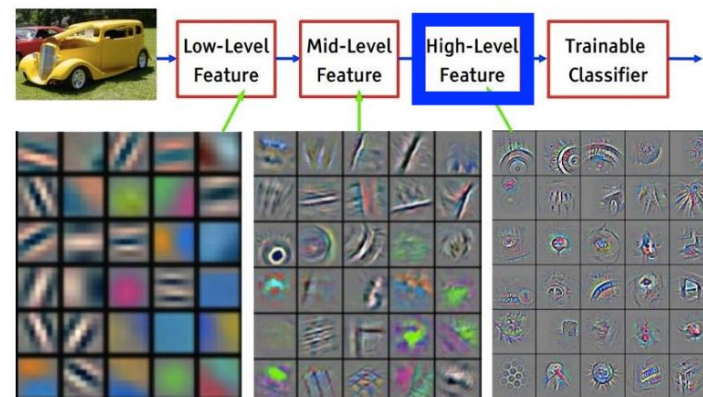
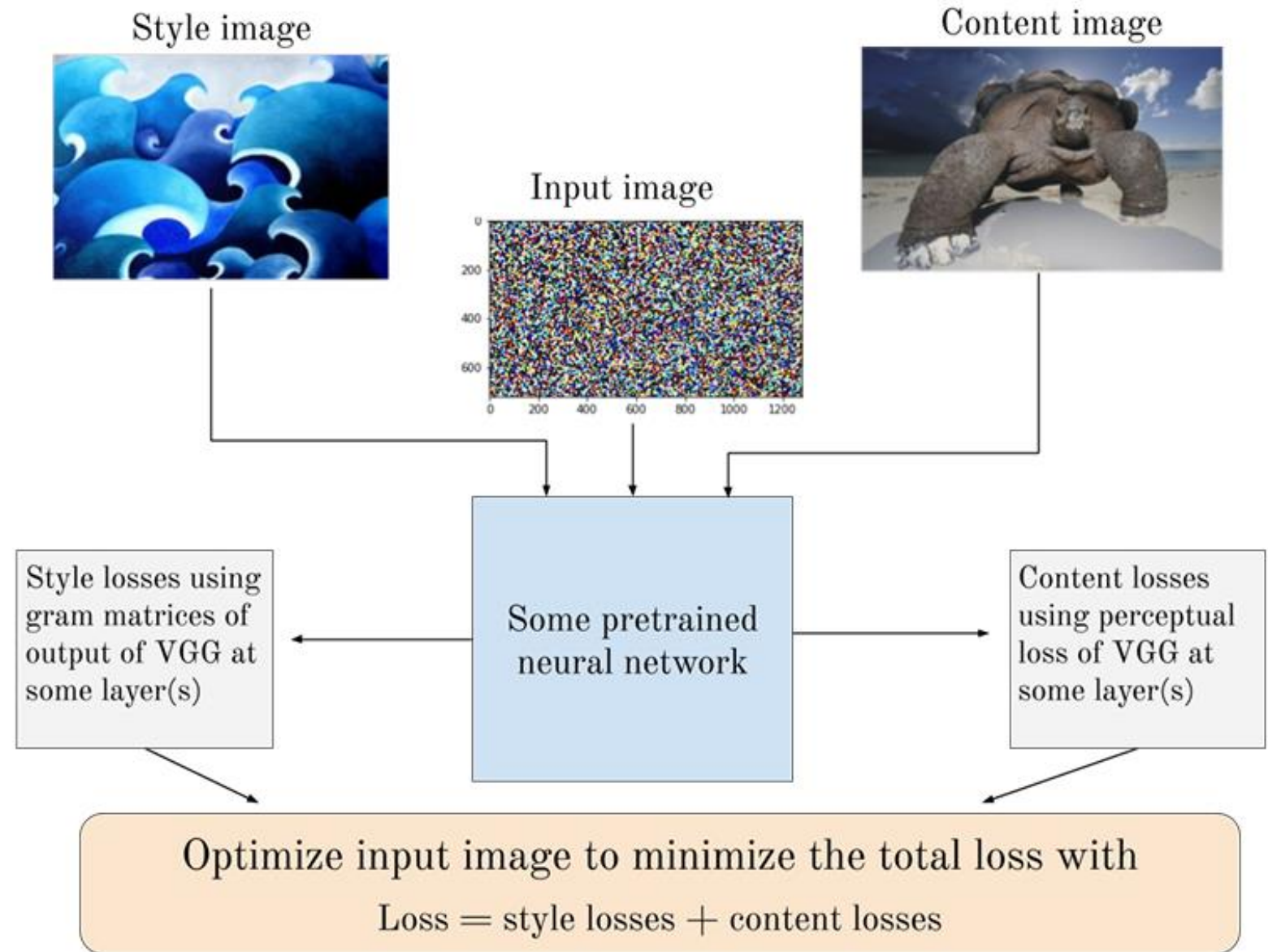


Figure Credit: Yann LeCun

Style Transfer

- Style transfer exploits this by running two images through a pre-trained neural network
 - Looks at the pre-trained network's output at multiple layers, and compares their similarity
 - Images that produce similar outputs at one layer likely have similar content, while matching outputs at another layer signals similar style
 - Enables us to compare content and style of two images, but it doesn't actually help us create the stylized image
- This is done by a second neural network - the transfer network
 - An image translation network that takes one image as input and outputs another image

Basic Setup

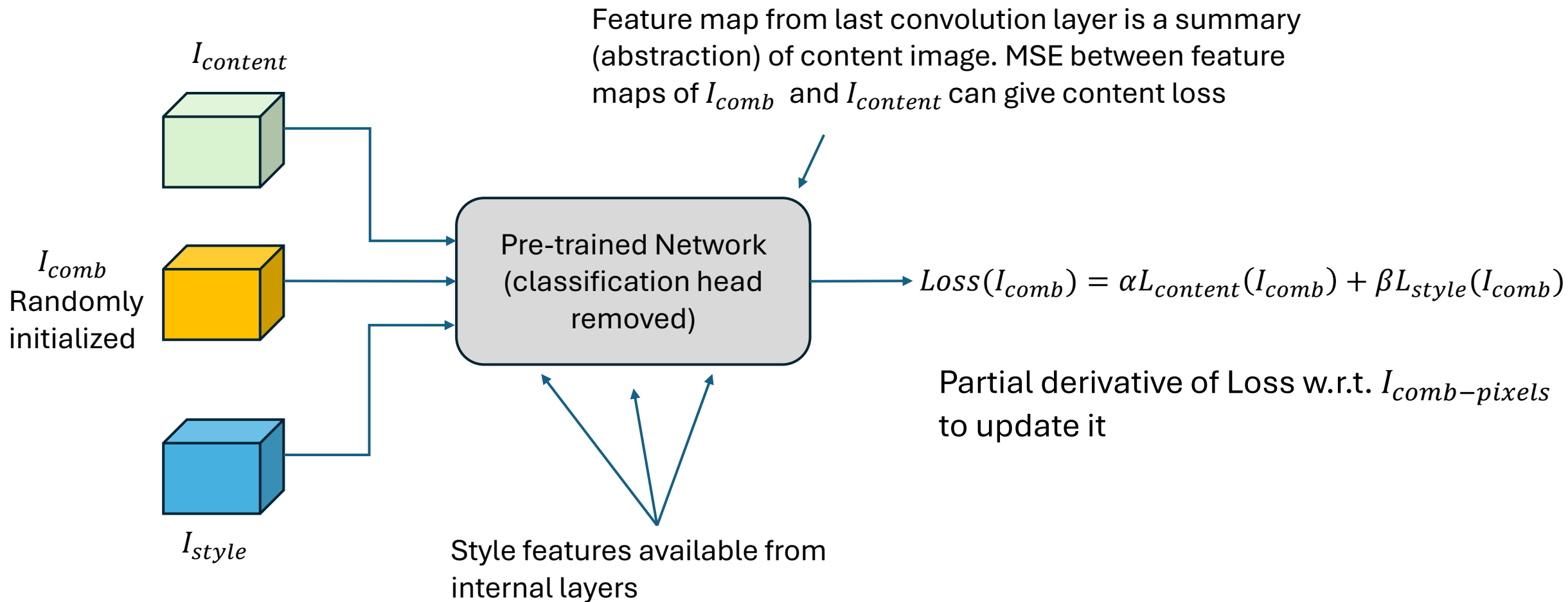


Basic Idea

- At beginning of training:
 - One or more style images run through pre-trained feature extractor
 - Outputs at various style layers saved for later comparison
- Content images fed into the system
 - Each content image passes through pre-trained feature extractor, where outputs at various content layers are saved
 - Content image then passes through transfer network, which outputs a stylized image
- Stylized (generated) image run through the feature extractor, and outputs at both the content and style layers are saved

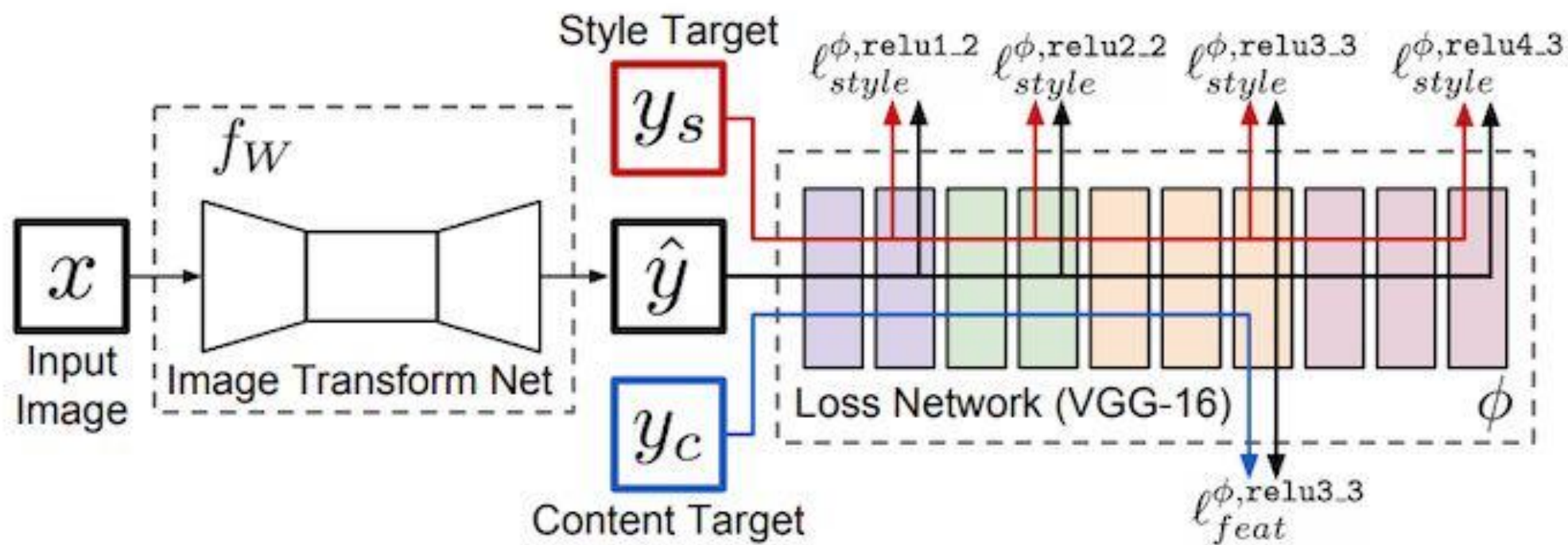
Basic Idea

- Model serves as a complex feature extractor
 - Need to leverage intermediate layers from the model
 - Use them to describe content and style of input images
- Input image transformed into representations
 - Have more information about content of image, rather than detailed pixel value
 - Features from higher levels of model can be considered more related to content of image
- To obtain a representation of style of a reference image, we use correlation between different filter responses



Basic Idea

- Quality of stylized image defined by a custom loss function that has terms for both content and style
 - Extracted content features of stylized image compared to original content image
 - Extracted style features compared to those from reference style image(s)
- After each step, only the transfer network is updated
 - Weights of pre-trained feature extractor remain fixed throughout
- By weighting different terms of loss function - can train models to produce output images with lighter or heavier stylization



Objective Function for Style Transfer

$$L_{total} = \sum_{l \in [L]} \alpha_l L_{Content}^l + \sum_{l \in [L]} \beta_l L_{Style}^l$$

- α_l and β_l are used to balance the style and content features

Content Loss

- Helps to establish similarities between content image and generated image
 - Intuitive that higher layers of model focus more on features present in image i.e. overall content of image
- Natural for a model to produce different feature maps in higher layers being activated in the presence of different objects
- Helps to deduce that images having same content should also have similar activations in the higher layers

Content Loss

- Calculated by Euclidean distance between intermediate higher-level feature representation F^l of generated image (x) and P^l of content image (p) at layer l

$$L_{content} = \frac{1}{2} \sum_{i=1}^{N_l} \sum_{j=1}^{M_l} (F_{i,j}^l - P_{i,j}^l)^2$$

- N_l is number of feature maps in layer l
- M_l is dimension (*height * width*) of feature map

Style Loss

- Total style loss across each layer expressed as:

$$L_{style} = \sum_{l \in L} w_l L_{style}^l$$

- w_l is weight of loss in layer l
- L_{style}^l is squared error between feature correlations expressed by Gram matrices of generated image and style image
- Gram matrix is a way to interpret style information in an image as it shows overall distribution of features in a given layer
 - Measured as amount of correlation present between features maps in a given layer

Style Loss

- Contribution of each layer in style information is calculated by the below formula:

$$L_{style}^l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{i,j}^l - A_{i,j}^l)^2$$

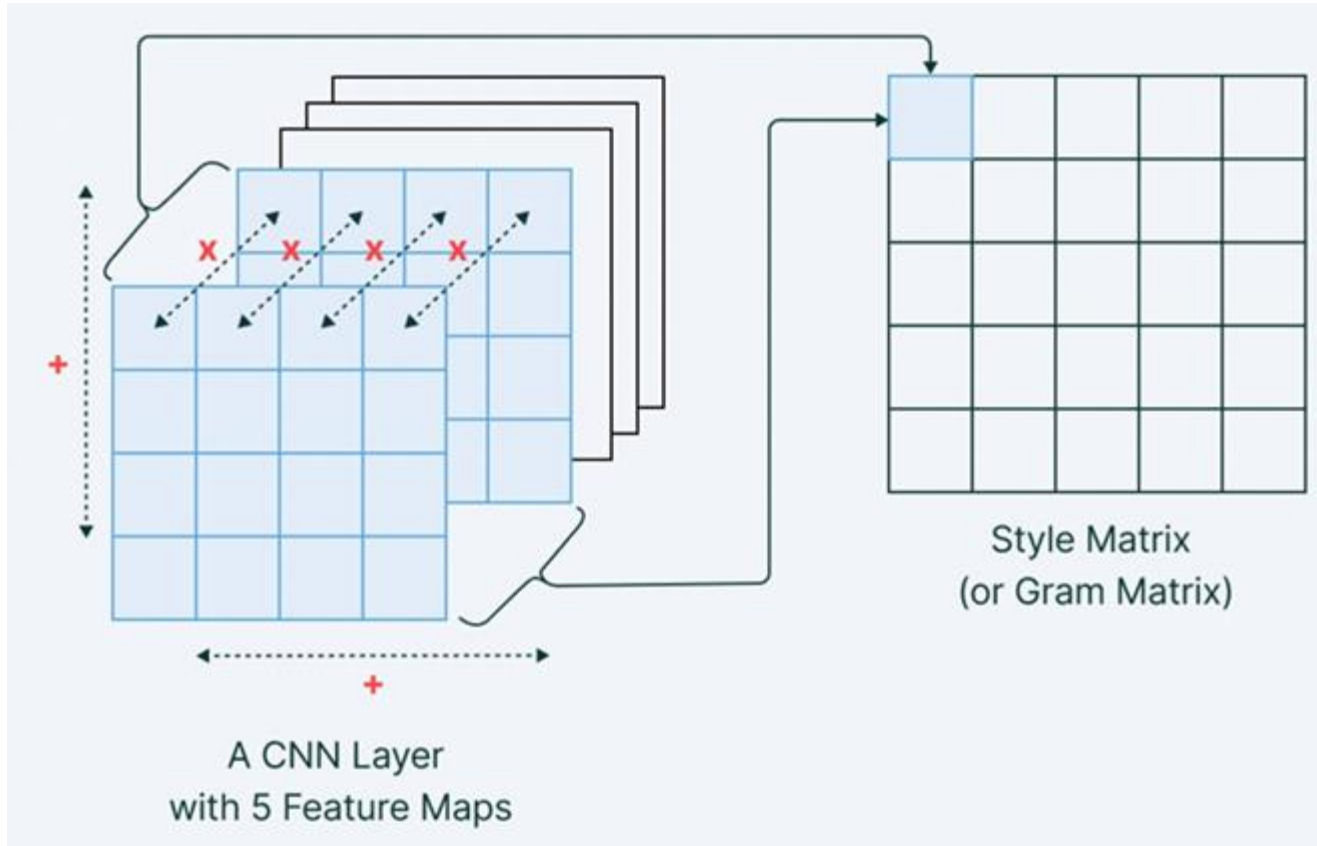
- $G_{i,j}^l$ is gram matrix of generated image and $A_{i,j}^l$ is gram matrix of style image at layer l
- N_l is number of feature maps in layer l
- M_l is dimension (*height * width*) of feature map



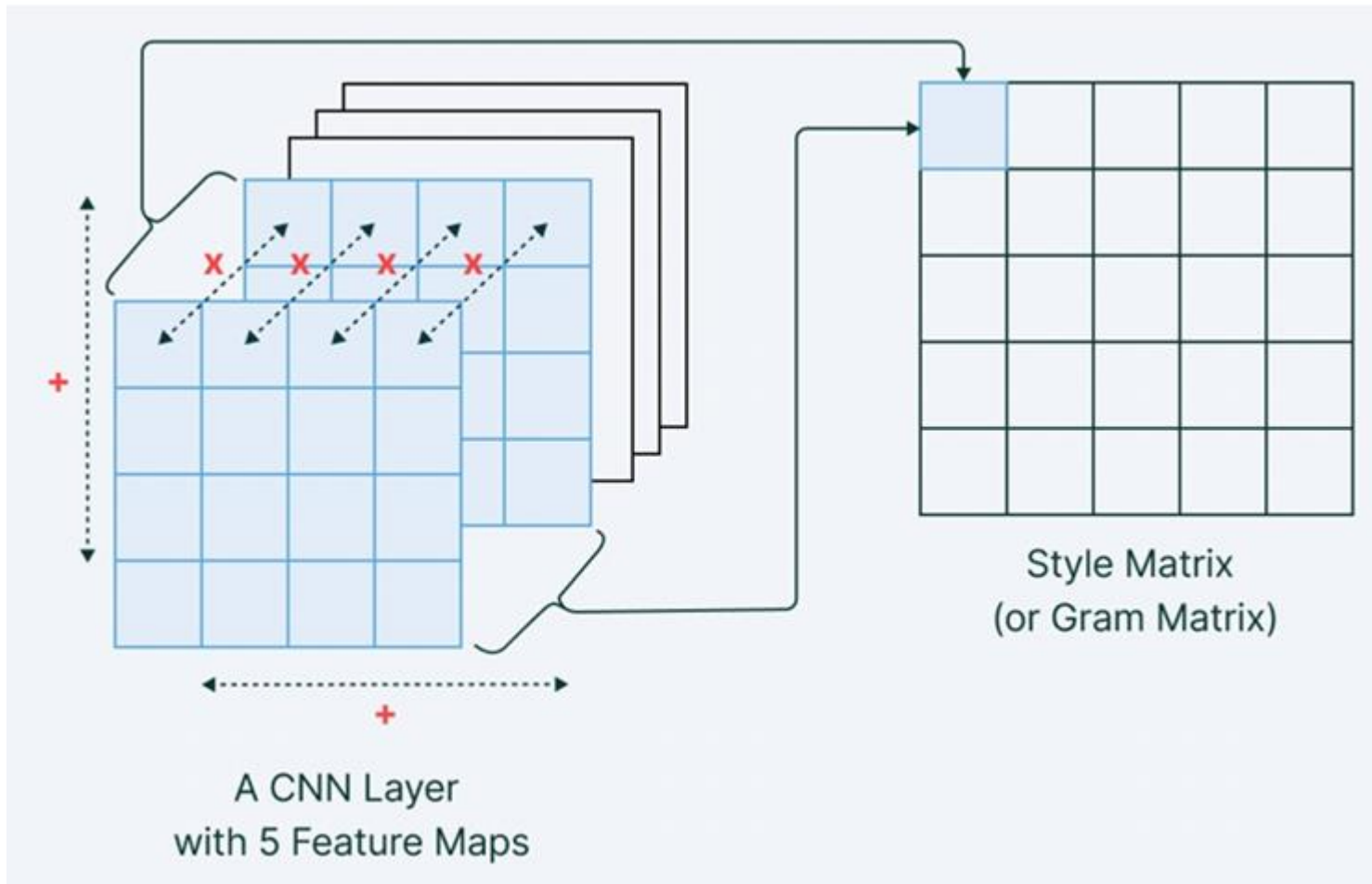
Style target

Gram Matrix

- **How to calculate:**
- Take activations of a layer
 - That layer will contain some convolution filters (e.g. 128), each one having its own activations
 - Convert each filter's activations to a (1-dimensional) vector
- Pick all pairs of filters
 - Calculate scalar product of both filter's vectors
 - Add scalar product result as an entry to a matrix of size #filters x #filters (e.g. 128×128)
- Repeat for every pair to get Gram Matrix
 - Gram Matrix roughly represents the texture of the image

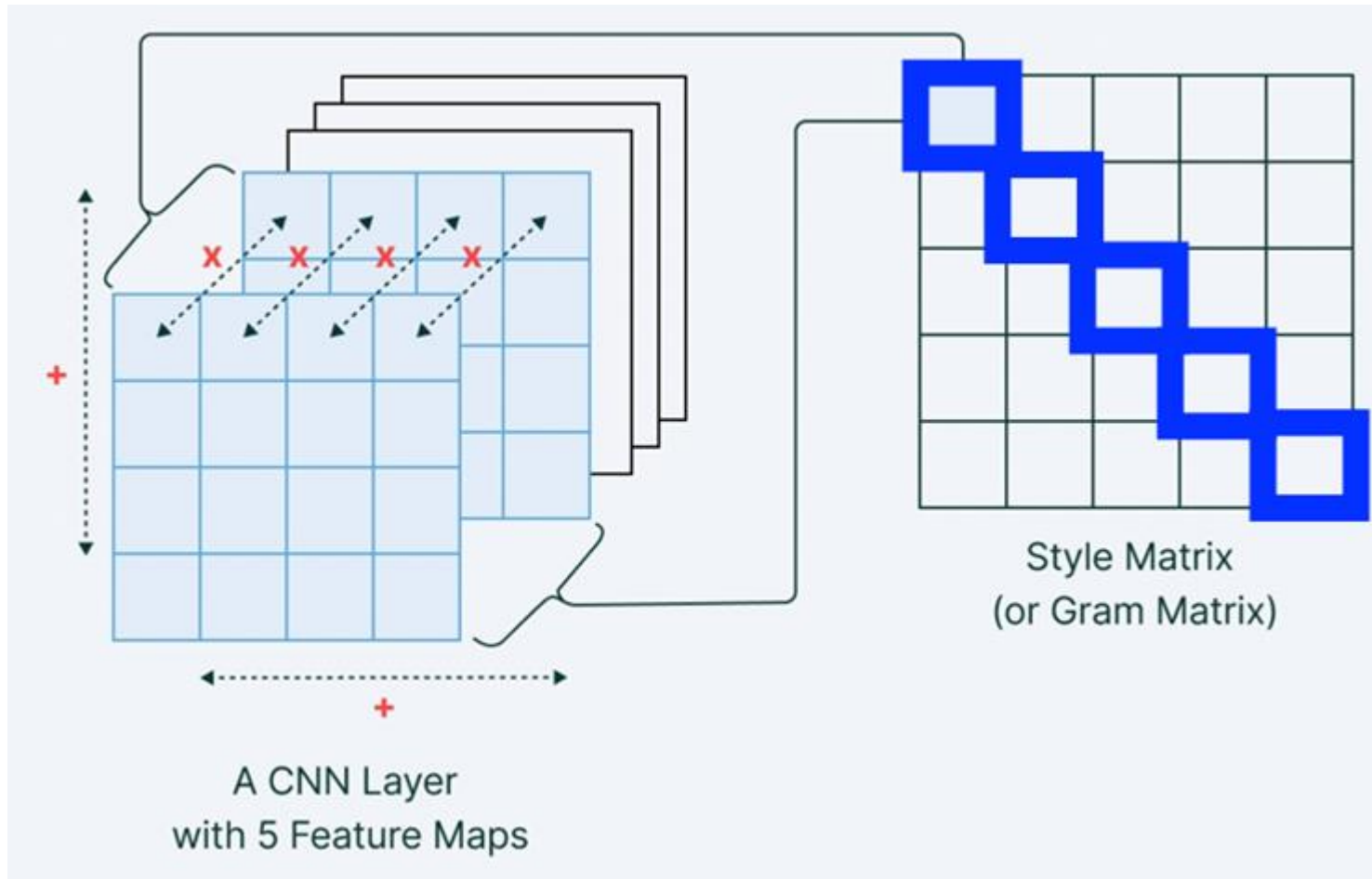


- For a layer, correlation computed between its feature maps (i.e. gram matrix)
- Each 2D map flattened into 1D – this removes structure information
- Dot product computed for each 1D vector with itself and others (larger values indicate greater feature co-occurrence)



N feature maps, each containing M values. What will be the dimension of the Gram Matrix?

- $N * N$



What should be the values on the diagonals of the Gram Matrix?

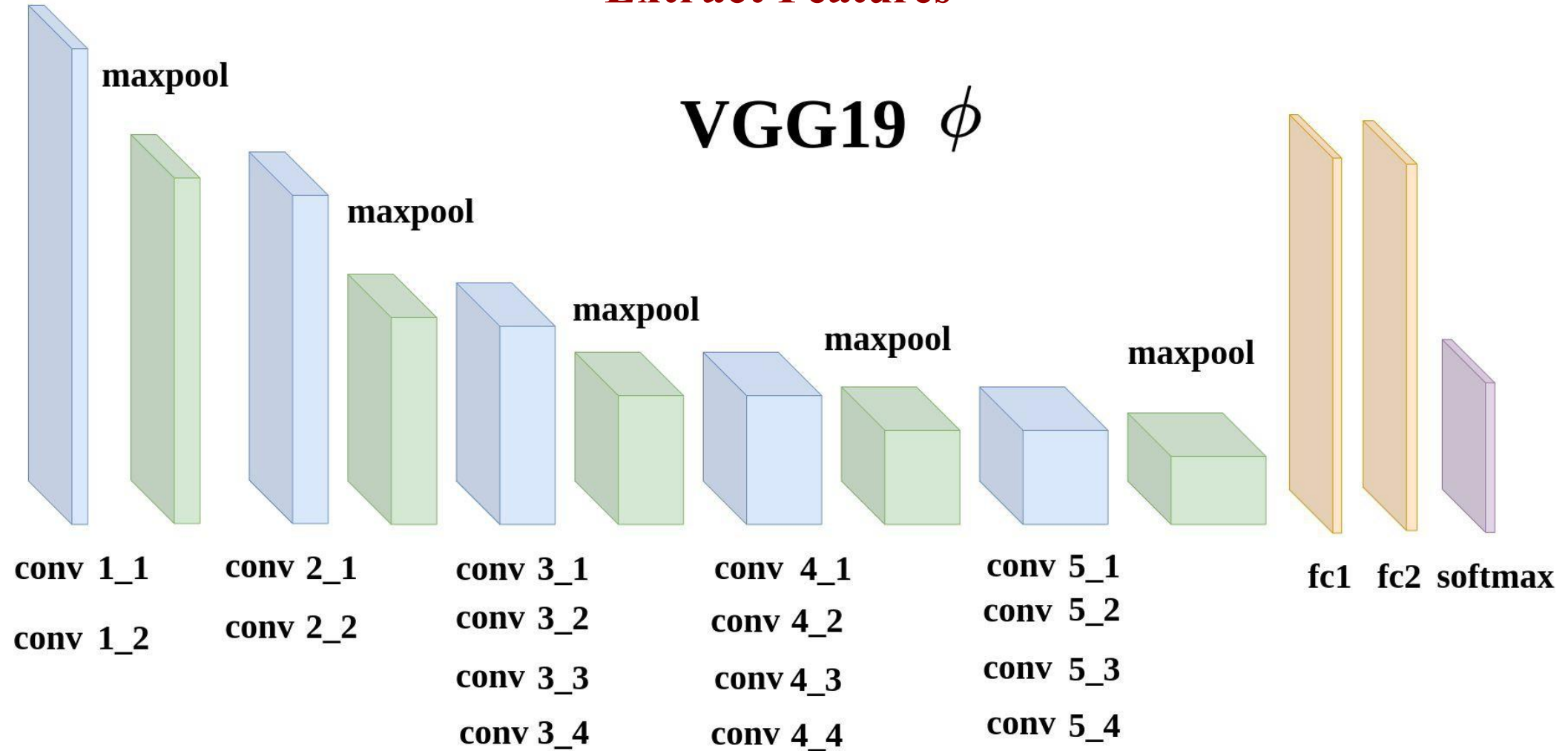
- 1 (reflects perfect match)

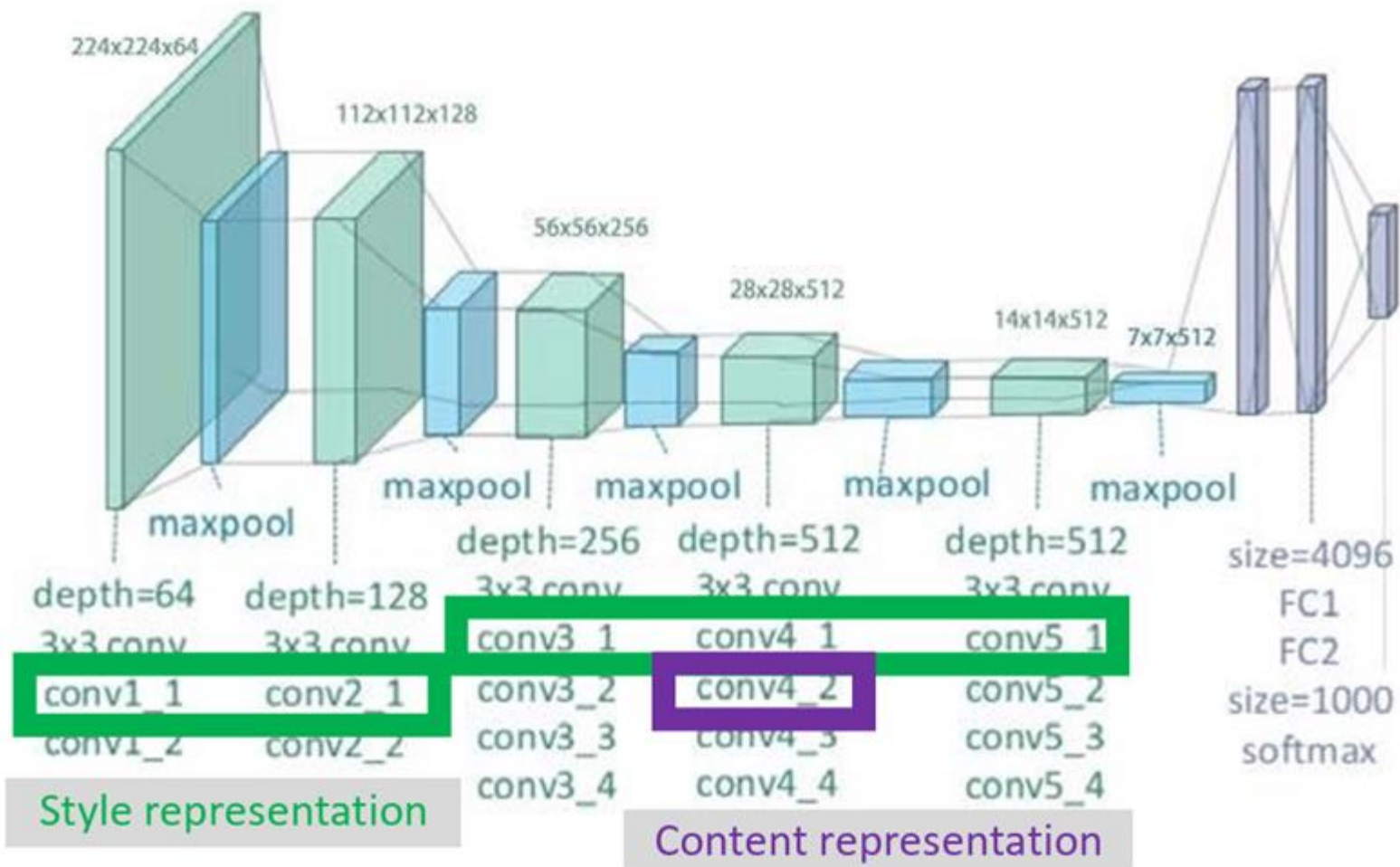
Implementation

- Use pretrained 19-layer VGG net as base network
- Assume two images are provided
 - *Content* image x_c , *style* image x_s
- Feed content image through VGG net and extract activations of last convolutional layer
 - These activations called the *content representation*
- Feed style image through VGG net and extract the activations of all convolutional layers
 - Transform each layer to a *Gram Matrix* representation - *style representation*
 - At beginning of training, one or more style images are run through the pre-trained feature extractor

Pre-trained Classifier to Extract Features

VGG19 ϕ

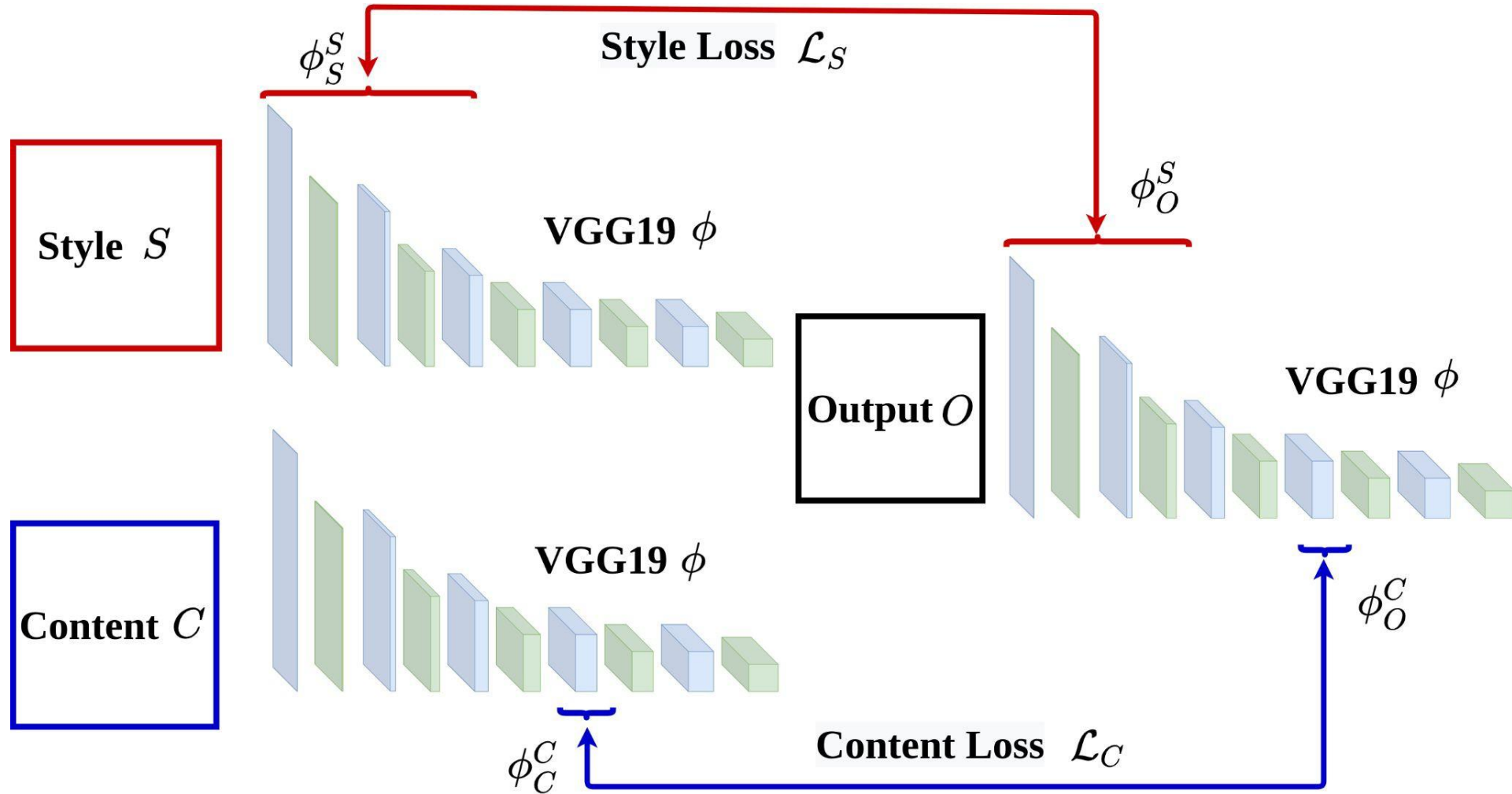




VGG-19 for feature extraction

- relu4_2 layer for content loss
- relu1_1, relu2_1, relu3_1, relu4_1, relu5_1 for style loss
- Default weight factor w_l set as 1.0 if not specified

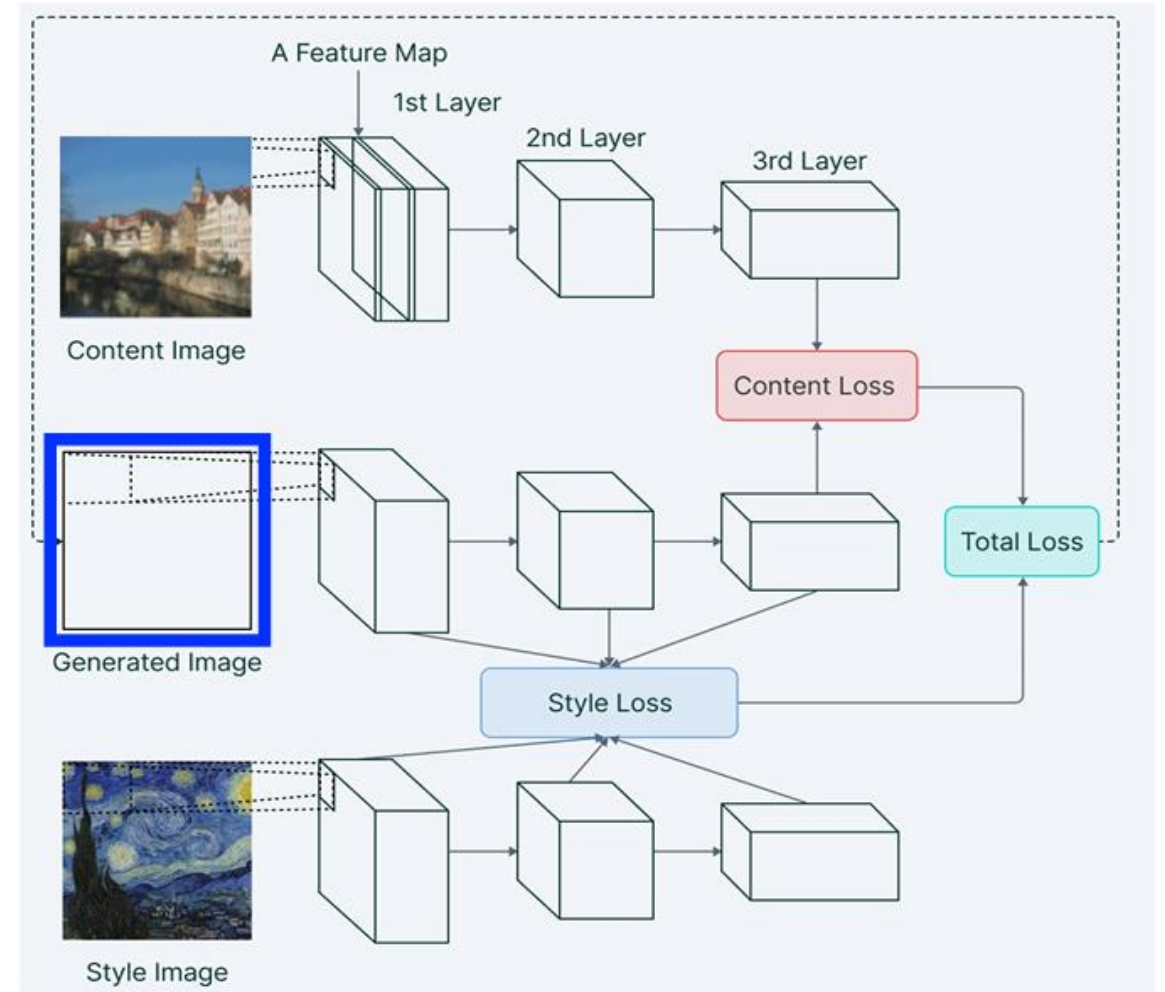
Style Transfer Key Idea



Implementation

- Create a new image of size of content image
 - Fill it with random white noise
- Feed that image through VGG to get its content representation and style representation
 - This step will be repeated many times during the image creation
- Make changes to new image using gradient descent to optimize a loss function
 - After each step, only transfer network is updated
 - Weights of pre-trained feature extractor remain fixed throughout

Approach: iteratively modify a random image guided by the content image and style image



Model architecture overview

- **Single style per model**
- A 2016 paper by Johnson et al was the first to train an independent neural network to stylize images in a single, feed-forward pass
- Large VGG16 models pre-trained on ImageNet are used for feature extractors, and a relatively small encoder-decoder network serves as the transfer network
- In this approach, a single transfer network is trained for each desired style

Model architecture overview

Multiple styles per model:

- In 2017, researchers at Google extended technique to allow:
 - A single transfer network to produce images in multiple styles
 - Blend more than one style together
- Inclusion of “conditional instance normalization” layers within network:
 - Produced stylized image could be conditioned on an additional model input
 - Take a content image as input, along with an additional vector telling network how much of each style to apply to the image
 - Ex., a model could be trained on a van Gogh, Picasso, and Matisse painting
 - When it's time to stylize an image, user can input $[1, 0, 0]$ for van Gogh, $[0, 1, 0]$ for Picasso, or $[0.33, 0.33, 0.33]$ for a blend of all three
 - Removes need to train and store multiple models for multiple styles
 - Provides creative freedom by letting users mix and match a continuum of styles

Model architecture overview

- **Arbitrary style transfer models:**
- A common characteristic of both single and multi-style transfer models is that they can only produce images in styles that they've seen during training
- A model trained on van Gogh's work cannot produce images like Picasso without retraining the entire network.
 - [Arbitrary style transfer by Huang et al](#) changes that
- Take a content image and a style image as input and perform style transfer in a single, feed-forward pass
- In essence, the model learns to extract and apply any style to an image in one fell swoop

Content Image



Style Image →



Artistic Style Transfer

Content Image



Style Image



Photorealistic Style Transfer